CS201 Lab 3

Data Structures and Algorithms

Shreya Agarwal Rachit Nimavat

Spring 2026

This lab builds directly upon your `IntVector` library from Lab 2. You will need your `vector.h`, `vector.c`, and `Makefile`. If your implementation of `IntVector` was buggy, you may start fresh by downloading a copy from the CS201 labs Github.

Part 1: Zero Gravity

You are given an array of integers. Write a function that moves all `0`s to the right end of the array while maintaining the relative order of the non-zero elements. You must do this **in-place** without making a copy of the array (i.e., only $O(1)$ extra space).

- Create a file `zerog.c` and implement the function: `void move_zeroes(int arr[], int n);`
- Write a `main` function to test it with some sample inputs in the same file.
- *Hint:* [1]

Example: Input: `[0, 1, 0, 3, 12]` Output: `[1, 3, 12, 0, 0]`

Part 2: Tera Kya Hoga Kaliya?

Our `IntVector` currently supports `append()` and `search()` operations. Implement a function `pop_index()` to remove an element at a specific index and shift the remaining elements to take up the space.

```
/* If index<0 or index>=size, do nothing.
Otherwise, remove the element at index and
shift remaining elements one step to the left. */
void pop_index(IntVector *v, int index);
```

Update the `vector.h` and `vector.c` files accordingly. Do you need to update your `Makefile`? With this new function, we can solve the famous **Josephus Problem**.

Gabbar Singh's N dacoits have failed a raid. Being a jovial fellow, he decides to play a game instead of shooting them all at once. He makes them stand in a circle, numbered 1 to N. He starts counting from person #1. He skips $k-1$ people and shoots the k-th person. The circle closes up (the body is removed), and he starts counting again from the person immediately next to the victim. This continues until everyone is eliminated. To savor the fear, Gabbar asks you, Sambha, to compute the order in which the dacoits will be eliminated.

Example: Input: `N=10, k=3` Output: `[3, 6, 9, 2, 7, 1, 8, 5, 10, 4]`

[1] Use two indices (pointers): `read_index` to scan the array and `write_index` to mark where the next non-zero should go.

Part 3: Budget Cuts

Our implementation of `IntVector` is great at growing, but terrible at saving money. In fact, our 'space complexity' is $O(m)$, where m is the maximum number of elements ever stored in the vector. This is because we only grow the vector when needed, but never shrink it. To fix this, implement **Automatic Shrinking** inside your `pop_index` function in `vector.c` and verify that your code for part 2 still works.

```
/* If index<0 or index>=size, do nothing.
Otherwise, remove the element at index and
shift remaining elements one step to the left.
If the size drops to less than a quarter of capacity,
shrink the capacity to half and print "Resizing from X to Y...",
where X and Y are old and new capacities respectively. */
void pop_index(IntVector *v, int index);
```

Questions to Ponder Over this Week

1. You'll use the `two-pointers approach` of part 1 in many problems. Internalize it and think about how can you use it to remove duplicates from a sorted array.
2. In part 2, what is the time complexity of your solution in terms of `N` and `k`? Remember this bound, and we will revisit it with Balanced BSTs.
3. Can you find the last dacoit standing without simulating the entire elimination process? Hint: [2]
4. In part 3, why not shrink when size hits half the capacity? Hint: [3]
5. For your `IntVector` library, convince yourself that `append` has **amortized time complexity** of $O(1)$. Moreover, at any point, the space used is $O(n)$, where `n` is the current size of the vector.
6. `make` is a powerful build tool. In writing lab-3 manual we have used it to automatically generate `lab-3.pdf` from `readme.md`.

[2] Use recursion and think about the position shifts after each elimination.

[3] think about what would happen if there were a series of `push` and `pop` operations at that boundary. This is called **Hysteresis**.